Thesis

Brain Simulation: Computation in Back Propagation Neural Networks

---

by **Scott MacGIBBON**

Supervisor: Dr Peter Nickolls

4 November, 1988

Part 0: Synopsis

Chapter 0: Synopsis

This thesis is concerned with the implementation of, and experimentation with, back-propagation neural networks.

Basic neural network models are explained, and the back-propagation model is described in detail in an algorithmic form.

The specification, design, and performance of the constructed software package, fishNET, are described. Equations for memory usage and approximate calculation and learning times are given.

Experimentation with learning and error rates shows that learning parameter and network parameter variations may have a large effect on learning time, but only a small effect on error rate. It is shown that networks which take longer to learn have a slightly lower error rate.

Introduction of casualties in both connections and neurons shows that knowledge in the network is distributed amongst the weights but localised in neurons, which act as feature extractors.

Finally, a complete printout of fishNET is included in the appendices, along with the dot-matrix encoded character data.

Part 1: Introduction

> *The shortest distance between any two points is always blocked.* ***Warren Monks, 1988***

Chapter 1: Introduction

The aim of this project is to implement a neural network of an unspecified type. I have chosen to implement the back-propagation model, due to Rumelhart et al. The work is intended to make qualitative statements on several facets of the model. These are: the model's ability to learn and its dependence upon parameters, the model's accuracy of operation and its dependence upon parameters, and finally the model's behaviour upon introduction of casualties of varying severity into the network.

In this chapter I intend to answer some fundamental questions about neural networks, namely: what their biological relevance is, why and how they solve problems, different models that have been suggested by other authors, and how these networks can be simulated in hardware and software.

In the next chapter I will discuss the back-propagation algorithm in detail, and the chapter following will contain a detailed implementation independent software specification for a software model.

1.1. Definition of Terms

Before I begin this discussion of neural networks, definition of a few terms will aid clarity [1].

A *connection* is a signal pathway between processing elements (or neurons), that correspond to the axons and synapses of biological neurons. Many connections form a neural network.

A *processing element* is a simple artificially simulated neuron that has a graded (analog) response. (From this point forward, I will draw a biological analogy and refer to processing elements as neurons.) It consists of a memory for its present state, and a transfer function that maps some function of the inputs to the output. The output is connected to many other processing elements' (or neurons') inputs.

A *weight* is a dynamic value (at least during learning) that determines the intensity of connections between neurons. Weights can be positive or negative (or zero,) indicating an excitatory or inhibitory (or no) connection. A weight is associated with a single input connection to a neuron.

1.2. So, What Is a Neural Network?

A neural network is a collection of graded response neurons (processing elements), representing an approximation to biological neurons, in some kind of feedforward/feedback network.

There are many different kinds of neural networks (Hecht-Nielsen [1] suggests that there are 13 main types), with the most common (or important) being the Hopfield, back-propagation (Rumelhart et al.), "improved" neocognitron

(Fukushima), and adaptive resonance theorem or ART (Carpenter and Grossberg). Each is particularly good at a number of tasks, and some have weak points. They vary considerably in complexity and power.

Neural networks are often described as a network of "collective decision" [2] circuits. They gain their power the same way the human brain does: many (many many) simple analog devices computing at the same time, while connected in parallel and continuously communicating with each other. The human brain has, by most estimates, 10^{11} neurons (100 billion!) with up to 10 000 interconnections per neuron. Imitating the performance of a human "biological neural network" with an artificial one is not and may never be feasible. However, some of the characteristics of the brain, such as the ability to learn or be taught (there is a difference here, as some networks teach themselves and some require a teacher) can be achieved. The brain often returns several possible solutions to a problem, with varying degrees of certainty. Artificial neural networks can do this. And both man and neural networks can infer the original from an image that is noisy, or showing only parts of the whole. Figure 1. Classic neural circuit. Figure 2. I/O response of simulated neurons. Figure 1 shows the classic feedforward and feedback neural circuit, and figure 2 demonstrates the sigmoid output response of a simulated neuron. These diagrams are referred to in a later section, "Why and how do neural networks solve problems?"

Simulations of neural networks involve several important simplifications to the behaviour of the individual neurons. It has been shown that analog neurons perform better than earlier two-state (binary neuron) attempts [2,17], but many features evident in biological neurons (much chemistry is discussed in [3]) are simply ignored in the most commonly used models of neurons, and analogies are made to voltages and currents. The model I will use for neurons is simple also, with the transfer function shown in figure 2 being the output from the inputs times the weights and summed. The relationship of the simple model to biology is shown in [4].

1.3. Neurological Relevance

Neurons themselves, in most models, are extremely simplified versions of the real biological article. So, it is the overall structure of a neural network that makes it relevant to neurology, not its individual components. This has been proven by many people; examples will be described below. Techniques for using neural networks in neurology take two approaches: the structure of the brain is discovered by experiment, and a network is built to verify this (there is suprisingly good correlation between the results of experiments such as this and the network), or a network of the appropriate model is built and taught and then the parts of the brain corresponding to the model are easily identified. Both methods, by dealing with small parts of the brain at a time, have yielded very good results.

Parts of the brain, specifically area 7a of the posterior parietal cortex in mon-

keys, have been successfully modelled by Zipser and Andersen [5] using back-propagation [20] learning in a feed-forward neural network, similar in form to the one intended for this project. The neurons respond to the location of the stimulus with respect to the eye and the position of the eyes to calculate the location of external objects. Even though the authors admit that there is no way that the back-propagation method is the only method used in the brain, it is very likely that a combination of Hebb-like [6,18] learning and the feeding back of errors would generate similar results. (Hebb was the first to suggest a biologically plausible method of learning. It is described in a later section.) They note also that all cortical connections in the brain have reciprocal feedback pathways, presumably for the feeding back of error signals.

Bear, Cooper and Ebner [3] have used neural networks in the reverse direction to study the primary visual cortex area 17 in adult cats. They used a neural network, determined theoretically how the neurons could behave, and compared this with the actual data. The theoretical model, then, enabled the researchers to sort out which of the possible hypotheses of brain function was most correct by experiment.

Frohn, Geiger and Singer [7] have used a model based not on rigorous maths, but, rather, neurological data, to account for the features in a mammal's visual system. They arrived at a 5 layer model, trained by a Hebb-like rule [6,18] that teaches itself. This allowed the authors to conclude that "the internal representation of a stimulus is the spatial activity pattern of a reciprocally coupled population of neurons," i.e. the knowledge of patterns in the visual system is distributed. Sun, Chen and Lee [19] also give a mathematical description of Hebbian learning of stereopsis in a neural network.

The importance of motion to the perception of structure in mammals has been simulated [8] with a three-layer back-propagation neural network, and the performance of the model was almost identical to that of man and monkey. The first and third layers of the model are designed to simulate neurons in a particular area of the brain, while it has not been accurately determined where the middle layer could lie.

Some models of neural networks, such as ART (due largely to Grossberg), are strongly based on structures found in certain parts of the brain. In fact, experiments with ART networks have led to a number of important predictions [9] of human brain structures that have been proven correct. Needless to say ART is an extremely complex form of network.

Other authors such as Lyon and Mead [21] have taken an entirely different approach. They argue that to build machines that operate like humans then the machines will need to "perceive" like humans. Hence they have built an electronic cochlea based around analog neuron-like components which operate like the human ear. This approach appears very fruitful, and I suspect we will see more machines designed about human form (cybernetics) in the future.

1.4. Why and How do Neural Networks Solve Problems?

As mentioned earlier, neural networks gain their power from the connection of simple analog devices in parallel. The human brain has roughly 10^{11} neurons, and each neuron can have up to $10\,000$ interconnections. Each connection can be an excitatory or inhibitory input or feedback, or an output connection to other neurons.

At present, it seems the largest neurocomputer[1] built to date has 10^6 neurons and interconnections totalling 1.5×10^6 in total [1]. This is, of course, far from the complexity of the human brain, but as we shall see it still offers a great deal of power in solving problems.

There are lots of common problems which cannot, as yet, be solved by standard serial algorithmic means (some suggest they can never be). For instance, recognizing the difference between friendly and enemy aircraft, a spoken language translator, or a system recognizing speech no matter who is speaking.

Human beings perform these tasks without any apparent difficulty, once they have learned to do so. Often, for many of these actions, there is no need to be explicitly taught. For all of these problems, we as man or woman can generate many examples of what the output should be for a given input, even though we cannot write down a fixed algorithm on how to do it.

But neural networks, without being programmed with an algorithm but taught instead, can already perform many of these tasks with 95% accuracy. For example, Kohonen [10] has built a phonetic typewriter, Fukushima [11] has used his "improved"[2] neocognitron model to recognise Chinese characters, and many authors have shown the ability of neural networks to remove noise and recognize patterns [7,11,12,23,24,25]. Many other implementations of this new non-algorithmic neural network programming are being carried out. A very good example is using neural networks as a signal analyser to detect subtle patterns in neuroelectric signals from brian data with "highly dimensioned noise" [26].

To understand how neural networks solve problems, we'll first try to get an intuitive feel for how they work, compared to a digital or Von Neuman machine that we are all, no doubt, familiar with already.

A digital machine takes an input, such as programs and data, and goes through a rigidly defined set of steps (instructions), branching if necessary, and producing an output of some kind, based upon program and data. On the other hand, neural networks or collective decision circuits, take inputs and move to minimize some function of the "computational energy" of the inputs, based upon the dynamic weights of the connections between the neurons. Some authors,

[1] Dedicated machines built to run models of neural networks appear to have acquired the jargonistic term "neurocomputers".

[2] In the latest writings I possess [11], Fukushima refers to the model he is working on as an improved version of his slightly older neocognitron model (circa 1984). Hence I have dubbed it an "improved" neocognitron, as no other official term was used in [11].

notably Hopfield and Tank [2][3]and recognized in [13] as relating to other physical systems as well, notably spin glasses, see neural networks as an energy reducing machine: imagine an n-dimensional surface of hills and valleys, with the computation beginning at some high point on the surface. As the computation procedes it will move to a valley, downhill (i.e. minimizing "computational energy"), until it is stable. The final answer then appears at the output neurons.

Consider figure 1 again. It contains several types of neurons, namely what we can call "principal" and "inhibitory" neurons (labelled by 'P' and 'IN' respectively). Looking at figure 2, imagine the response of the 'P' neurons follows the solid line, and the response of the 'IN' neurons follows the dotted line. Then, the response of the circuit to an input will depend upon the weights on the connections between neurons. As you can see, the deviation from zero of an input to any neuron (and the input is the sum of the outputs of the previous neurons times their connection weights) will tend to drive it strongly towards a positive or negative output. All the neurons "calculate" at the same time, so the behaviour of a neural system (especially a very large one) is not trivial, and can't be written as a serial algorithm.

1.5. Types of Neural Networks

There are many different types of neural networks. Some are particularly suited to some types of problem, some are suitable for many types of problem. Some can "learn" without supervision, others require a teacher. Most of these neural models are grounded upon Donald Hebb's [6] theory of synapse modification. He indicated that a plausible explanation for learning would be for connections between neurons to strengthen if the activity of both ends of the connection increased (ie., reinforce the connection) and weaken the connection if the ends weaken in activity (ie., inhibit the connection). To various extents, all these models aim to implement "Hebbian" learning by one method or another. (Hebb never mathematically quantified his thinking. A good approach to doing just this is given by Linsker [18].) Examples of several types of neural networks appear below.

1.5.1. The Hopfield Network

The Hopfield network shown in figure 3 is the simplest type of network, yet it is suprisingly powerful.

Figure 3: Hopfield network It has a feedback structure, and it cannot learn (although a technique for teaching feedback networks has been suggested by Atiya [14]). The selection of weights depends upon the problem at hand, for example classic computer science problems such as the Travelling Salesman Problem (TSP), the book stacking problem, and a symmetry detector. All these

[3]**

problems can be computed after the appropriate weights are set up in the feedback structure.

Consider the Travelling Salesman Problem, or TSP. A salesman is required to visit a number of cities, by the shortest possible route. Imagine that there are 30 cities on the tour. Comparing the neural network solution to the serial solution shows some staggering results!

Let's see the difference between a brute force approach, a good serial algorithm, and a Hopfield neural network solution. Based upon estimates from [15], probably 10^{30} comparisons would be needed for a brute force approach. This is obviously not computable. An excellent serial algorithmic estimation approach would require roughly $1\,500$ comparisons in the best case (but probably many more). But a Hopfield neural network, it has been found, will select one of the best 10^7 solutions (and this is as good as or better than the algorithmic approach described above) in a single convergence of the network, or several time-constants of the ciruit of figure 3. Quite a difference.

1.5.2. The Back-Propagation Network

This is another "classic" neural network, along with the one described above (they appear to be the most commonly implemented in neurological experiments). The inventors and main developers [1] are considered to be Werbos, Parker and Rumelhart. It has a feedforward structure, as shown in figure 4.

Figure 4: A typical back-propagtion network This network can be taught [20], ie strengths of connections may be varied within a rigidly defined set of rules called a "learning algorithm", rather than being set by hand. The next chapter explains the back-propagation algorithm in detail.

The network learns by comparing the network output with the desired output, for a fixed input. The error is used to change the weights, then another input/output pair is presented. The process continues through the samples as many times as necessary until the error falls below a certain threshold. How many times each input/output pair needs to be presented is a matter of conjecture, and one of the things I'm going to investigate later in the project.

By itself, back-propagation networks are not neurologically plausible. However, the propagation of errors back through the network by feedback is known to occur in the cerebal cortex.

Like all other neural networks, this one is redundant (ie knowledge is distributed) and so behaves sanely when casualties in processing elements occur. Once again, exactly how robust a network is is for me to investigate later in the project.

Back-propagation networks are good for solving problems such as speech synthesis and adaptive control.

1.5.3. The "Improved" Neocognitron

The "improved" neocognitron, along with the next model discussed, falls into that category of neural networks that are more neurologically sound in their design. They both contain more complex neurons, with complex feedforward and feedback connections.

Fukushima [11] has used his model to recognise Chinese characters, a task that is readily performed by over 1 billion Chinese every day but was yet to be done reliably by machine. It is largely immune to those bugbears of artificial neural networks, namely rotation, translation, and changes in scale. One of the most astounding properties of this model is its ability to shift "attention", just like a human, from one pattern to another for an input that contains several recognisable patterns at the same time. (See [11], which has examples.)

Figure 5: An example of an "improved" neocognitron

Figure 5 shows us the nature of the network: feedback and feedforward paths, with dynamic gain and threshold controls forming part. There are fixed and dynamic connections between neurons.

The model functions as follows: As patterns are recognized by the network (by itself ie., without a teacher), the top or "recognition" layer of neurons becomes active. The output of this layer is fed back down to the lower stages, which, by varying the gain, enforce the forward flow of the recognized pattern. After the network has learnt to recognize the presented patterns, by simply breaking the feedback path for an instant the "attention" is switched to another recognizable pattern, and a new pattern will be recognized at the output. Needless to say, this network is too complex to implement in the short time available, so this is as thoroughly as it will be discussed.

1.5.4. The Adaptive Resonance Theory Model

The Adaptive Resonance Theory (or ART) model is the most complex one discussed here [9].

ART systems belong to a group of neural networks which fall into a category of "competitive learning" models. It has been used for visual pattern recognition, speech perception, and radar classification.

Even though it is the most complex model examined here, it can be simply described as a two layer network with gain control. There is a distinction between "short" and "long" term memory, with memory decaying slowly over time (like a human brain, if some things are not used all the time they are gradually forgotten…) There is extra circuitry as well for resetting the second layer (ie disabling its reinforcing action) when there are sufficiently large mismatches between the two levels.

1.6. Simulating Neural Networks in Hardware

As explained earlier, neurons have a graded response. Hence it is possible to simulate a neuron with a transfer function like that of figure 2 as an amplifier with a similar response. It would be possible, if one stretched one's imagination a little, to build a huge multi-million neuron "brain" from op-amps and resistors, similar in structure to the network shown in figure 1. However, this is not a particularly sensible aim, for instance how do the weights change automatically? (Who twiddles the knobs on the variable resistor as the network learns?) For *small* problems, this technique could be used.

Figure 6: A special machine for the TSP For example Hopfield and Tank [2] devised such a contraption shown in figure 6 using a Hopfield network that could solve the TSP for however many cities it was wired for (I don't think they actually built it). The output is indicated by a globe lit for the appropriate city (column) in the appropriate order (row) of the salesman's trip. The device's weights are set so that only one globe can light fully in any row or column. Needless to say after that example the discrete approach to neural networks is not particularly popular.

There have been some attempts at analog ICs with variable internal weights (one such is described in [2]). The ones built have functioned successfully as associative memorys. However the density and accuracy of VLSI analog ICs is low, so this approach has not been terribly fruitful.

A novel approach to hardware simulation has been described by Murray and Smith [16], whereby they use digital ICs which use pulse stream arithmetic. Streams of pulses of varying frequency are gated within the connections. This gives a situation near to biology, where a neuron that is *"on"* produces a regular train of digital pulses (the frequency dependent upon its *"on"*ness), while a neuron that is *"off"* produces nothing. This technique is still in the experimental stage at the date of publication, but it could prove possible to efficiently perform significant calculations in this way in hardware.

Another novel approach is described by Vidal [22]. He proposes a neural network implemented in programmable logic, using purely digital techniques, and argues that possibly digital (Boolean) techniques offer a good solution to network problems. This is an example of how varied the models for both hardware and software are, and how much there is that is still undiscovered and undecided in this field.

The perfect hardware for the parallel operation of a neural network is not planar silicon. Most authors suggest that three dimensional biological materials are infinitely better suited to parallel processing than a two dimensional substance. However optical computers, which are parallel by their very nature, could offer processing solutions in the long term.

It is now easy to understand why most neural networks are simulated in software. Most of the references given refer to software models as they are more

flexible, though slower. The next section deals briefly with this.

1.7. Simulating Neural Networks in Software

As I stated at the start of this chapter, this project is intended to produce a working neural network of the back-propagation model [20]. Simulation of the parallel activity of the neurons is possible by imagining the operations occuring in discrete time steps, and calculating one row of neurons at a time. Of course, this technique works best when signals flow in only one direction, such as in back-propagation models, because then one can calculate the first layer, then the values from the first give the second, etc.

The simulation of other networks such as the "improved" neocognitron will not be considered here, but I suspect it would not be much more difficult: possibly requiring calculations of layers with smaller time steps due to the more complex interactions between layers.

The next chapter describes the mathematics behind the back-propagation algorithm to be implemented, based largely on [20] by Rumelhart, Hinton and Williams.

Chapter 2: The Back-Propagation Model

This short chapter aims to describe, in a step-by-step way, the mathematical steps behind an implementation of the back-propagation model. The maths in the first sections is based upon Rumelhart et al., [20]. My interpretation of the maths, the proposed method of implementation, appears in the last section of this chapter. As in the last chapter, a biological analogy is drawn and processing elements are referred to as neurons.

2.1. The Maths Behind the Model

Based upon figure 7, which shows a feedforward neural network, the following mathematics holds:

Figure 7: A feedforward neural network

Consider two neurons, i and j, with neuron i being a lower layer (that is closer to the input) than neuron j. Let the inputs to a neuron j be x_j, and the outputs of a neuron i be y_i. Let the weight of the connection between i and j be w_{ji}. Then the total input x_j into a neuron j is a linear function of the outputs y_i of the units connected to j and of the weights w_{ji}:

$$x_j = \sum_i y_i w_{ji}. \qquad (1)$$

Extra input or "bias" can be added to a neuron, say j, equivalent to a threshold of the opposite sign of its weight, assuming the input is 1. The bias is treated just like any other y, ie $y_{\text{bias}} = 1$, but $w_{j,\text{bias}}$ gives the negative threshold for j.

Each neuron j has a non-integral output y_j, which is a non-linear function of its input x_j :

$$y_j = \frac{1}{1 + \mathrm{e}^{-x_j}}. \qquad (2)$$

The input/output function doesn't need to be the sigmoid response, any non-linear function with a bounded derivative will do.

Let c be an index of cases, j be an index of output neurons, y be the actual state of an output neuron, and let d be the desired state (or expected answer). The total error, E, is defined as

$$E = \frac{1}{2} \sum_c \sum_j (y_{j,c} - d_{j,c})^2. \qquad (3)$$

The whole aim of this technique is to minimize the error, ie. make the expected and actual outputs more similar. To minimize E by gradient descent, we need the partial derivative of E with respect to each weight in the network. So from (3),

$$\frac{\partial E}{\partial y_j} = y_j - d_j. \qquad (4)$$

And

$$\frac{\partial E}{\partial x_j} = \frac{\partial E}{\partial y_j} \frac{\partial y_j}{\partial x_j}. \qquad (5)$$

From (2),

$$\frac{\partial y_j}{\partial x_j} = y_j(1 - y_j). \qquad (6)$$

Hence

$$\frac{\partial E}{\partial x_j} = \frac{\partial E}{\partial y_j} y_j(1 - y_j). \qquad (7)$$

So we can calculate how a change in input x to an output neuron will affect the error. However, the input to an output layer neuron is a linear combination of the outputs from the lower layers and their weights. So we can compute what the effect on the error would be for changes in lower states and weights:

$$\frac{\partial E}{\partial w_{ji}} = \frac{\partial E}{\partial x_j} \frac{\partial x_j}{\partial w_{ji}} = \frac{\partial E}{\partial x_j} y_i. \qquad (8)$$

Then

$$\frac{\partial E}{\partial x_j}\frac{\partial x_j}{\partial y_i} = \frac{\partial E}{\partial x_j}w_{ji}, \qquad (9)$$

and taking into account all connections from i,

$$\frac{\partial E}{\partial y_i} = \sum_j \frac{\partial E}{\partial x_j}w_{ji}. \qquad (10)$$

The procedure outlined above can be repeated for every layer below the output layer, computing $\frac{\partial E}{\partial w}$ as we go.

This brings us to the philosophies that can be employed to modify the weights.

2.2. Modification of Weights

The simplest scheme of weight modification is to modify them as we go for every input/output pair. (To change w, modify by $\Delta w = \frac{\partial E}{\partial w}$.) This method doesn't require $\frac{\partial E}{\partial w}$ to be stored for each pass.

Another method, used by Rumelhart et al. [20] is to accumulate $\frac{\partial E}{\partial w}$ over all the input/output pairs before changing the weights. The simplest version of this method is to make

$$\Delta w = -\varepsilon\frac{\partial E}{\partial w}, \qquad (11)$$

where ε is a constant of proportionality and $\frac{\partial E}{\partial w}$ has been accumulated over all cases. An alternative version which apparently offers speed improvements is to use a proportion of the previous Δw,

$$\Delta w(t) = -\varepsilon\frac{\partial E}{\partial w(t)} + \alpha\Delta w(t-1), \qquad (12)$$

where t is a count of the number of times all input/output pairs have been presented and α is an exponential decay factor. This is the method that I will use in the software model of the back-propagation network.

2.3. An Algorithm for The Back-Propagation Model

From all this mathematics, we can now explain the back-propagation model algorithmically. Two algorithms are presented. I have found the first to cause networks to converge faster, but the second may prove useful in some circumstances. In the first algorithm $\frac{\partial E}{\partial w}$ is accumulated over all cases, and $\Delta w(t)$ is calculated after all input/output cases have been presented. In the second algorithm, $\frac{\partial E}{\partial w}$ is used to calculate $\Delta w(t)$ after every input/output case.

The first algorithm is:

1. Set up random weights between neurons.
2. Set $\sum \frac{\partial E}{\partial w}$ to zero, and set E_{TOTAL} to zero.
3. Input data sample and compare output with expected answer. For all output neurons, if output differs from the expected value by more than 0.2, increment E_{TOTAL}.
4. If dealing with the weights for the top layer, calculate $\frac{\partial E}{\partial y}$ from (4). Otherwise, calculate $\frac{\partial E}{\partial y}$ from (10).
5. Calculate $\frac{\partial E}{\partial x}$ from (7).
6. Calculate $\frac{\partial E}{\partial w}$ from (8) and add to $\sum \frac{\partial E}{\partial w}$.
7. If there are more input/output data pairs, go to 3. again.
8. Calculate $\Delta w(t)$ from (12), and save $\Delta w(t)$ as $\Delta w(t-1)$ for next pass.
9. Apply $\Delta w(t)$ to change weights, w.
10. If $E_{\text{TOTAL}} = 0$, we are finished and the network has "learnt". Otherwise, go back to 2. and repeat the procedure for the full set of data pairs.

The second (alternative) algorithm is:

1. Set up random weights between neurons.
2. Set E_{TOTAL} to zero.
3. Input data sample and compare output with expected answer. For all output neurons, if output differs from the expected value by more than 0.2, increment E_{TOTAL}.
4. If dealing with the weights for the top layer, calculate $\frac{\partial E}{\partial y}$ from (4). Otherwise, calculate $\frac{\partial E}{\partial y}$ from (10).
5. Calculate $\frac{\partial E}{\partial x}$ from (7).
6. Calculate $\frac{\partial E}{\partial w}$ from (8).
7. Calculate $\Delta w(t)$ from (12), and save $\Delta w(t)$ as $\Delta w(t-1)$ for next pass.
8. Apply $\Delta w(t)$ to change weights, w.
9. If there are more input/output data pairs, go to 3. again.
10. If $E_{\text{TOTAL}} = 0$, we are finished and the network has "learnt". Otherwise, go back to 2. and repeat the procedure for the full set of data pairs.

The next part, Software, deals with the implemented software for the back-propagation algorithm, where both the algorithms above have been implemented.

Part 2: Software

Allen's Law: *Everything is more complicated than it appears.*

Chapter 3: Software Specification

This chapter outlines the user interface and the functions to be performed by the software model of a back-propagation neural network. In the following specification, a *message* will convey some information on program operation to the user, a *warning* is a non-catastrophic signalling to the user, while an *error* will cause a termination of the program after an error message is printed.

The specification as it is presented here has been fully implemented in the software package, called fishNET. The design used to implement this specification is described in Chapter 4: Construction, and its behaviour is examined in Chapter 5: Performance.

3.1. The User Interface

Operation of the program requires a number of parameters and fishNET offers a number of different ways of saving and viewing the data about network functions and the results of learning. The program accepts command line inputs of names of files which contain network data, or can accept input from the keyboard if a data file is absent. Both methods of input (file and keyboard) will be described below.

3.1.1. Selecting the Parameters – Configuring the Network

The user is able to select the size and shape of the network, as well as the parameters for use during learning. Entry is via an input file specified on the command line, or by answering questions about the individual data items if no command line file name is given.

The information needed to run the program is:

- A comment or description of the problem for saving with the network.
- The number of layers in the network, and the number of neurons in those layers.
- The names of the files containing teaching input data and expected data.
- The name of the file containing data for use in execution and the name of the output file.
- The name of the file where the network is to be saved.
- The learning parameters α and ε.

- The nominal output width of the output layer of neurons for output formatting.
- The maximum number of input/output pair learning sweeps required, if a limit is desired.

If no command line file name is specified, the program will query the user via the keyboard. This is the default operation.

3.1.1.1. Keyboard Input Format A question is asked via the screen for every parameter listed above, and the answers are accepted through the keyboard. The user enters a return after typing the parameter being considered.

3.1.1.2. Configuration File Format The format of a configuration file is described below. The symbols $\langle\rangle$ are used to delimit user required data. All information in double quotes " " are tokens and must be inserted *exactly* as they appear otherwise an error will occur. (The error message is 'appropriate token not found'.) All tokens and data must also appear in the order shown below. The exception is any piece of data which is surrounded by $\{\}$ brackets. These data items are optional. If they are used, however, please note the quoted and bracketed tokens and data items necessary.

"layers"	< n>				
"neurons"	< a_0	a_1	a_2	…	a_n-1>
"output width"	< m>				
"alpha"					
"epsilon"					

"sample in"	path
"sample out"	path
"execute in"	path
"execute out"	path
"network save"	path
"max sweeps"	< q>

Simply place the desired value next to the appropriate token. Tokens must be in order. For the "neurons" token, the order of the numbers $a_0 \rightarrow a_{n-1}$ is taken as the number of neurons in the layers $0 \rightarrow n-1$. Hence the example | layers | 3 | | | | — | — | — | — | | neurons | 13 | 20 | 5 | means that there are 13 neurons on the input (layer 0), 20 in the intermediate layer (layer 1), and 5 in the output layer (layer 2). The tokens "alpha" and "epsilon" refer to the learning parameters α and ε. The file name following the "sample in" token is the file used as training input for the network, and the file name following

the "sample out" token is the expected output file, used for comparison and calculating the error. The "execute in" and "execute out" tokens precede the file names for use in operating the network once it is taught and the final destination of the output. The token "network save" comes before the name of the file where the network is to finally be saved once it has been taught, or the program terminated. Finally, "max sweeps" is the maximum number of training sweeps that will occur. Once this number is reached, (and it will not be reached if the network is completely taught,) the network is saved and execution begins. Formats of data files are shown below. Command line options used to specify a configuration file instead of keyboard entry are found in section 3.1.5. Command line options. ### 3.1.2. Using a Pre-Made Network The user is able to load a network file which contains all the data necessary (especially weights) for operation of the network, or continued teaching, or both. The code is designed so that if it is halted (a Ctrl-c in MSDOS, or a BREAK or DEL in Unix) the network's present status and elapsed t are stored into a network file. The network is also saved when it has been fully taught, or the maximum time has elapsed (depending upon flags set, so see 3.1.5. Command line options). #### 3.1.2.1. Network File Format The format of a network file is described below. The symbol format is the same as the previously described configuration file format (3.1.1.2). | "layers" | < n> | | | | | — | — | — | — | — | — | | "neurons" | < a_0 | a_1 | a_2 | … | a_n-1> | | "weights" | < b_0 | b_1 | b_2 | … | b_m-1> | | "output width" | < m> | | | | | "alpha" | | | | | | | "epsilon" | | | | | | "sample in" | path | | — | — | | "sample out" | path | | "execute in" | path | | "execute out" | path | | "start time" | < q> | | "learn time" | < r> |

Place the desired value next to the appropriate token. Tokens must be in order. For the "neurons" token, the order of the numbers $a_0 \rightarrow a_{n-1}$ is taken as the same as the number of neurons in the layers $0 \rightarrow n-1$. Data following the "weights" token behaves identically: if there are a_0 neurons in layer 0, and a_1 in layer 1, then there will be $a_0 \times a_1$ weights between layer 0 and layer 1. The first a_1 weights are the strengths of connections between the first neuron in layer 0 and the neurons in the layer above. The second a_1 weights apply to the second neuron's connections to the layer above, etc., until the a_0th a_1 weights apply to the last neuron in layer 0. The pattern is repeated for all layers (note that there are no weights associated with the output $n-1$th layer).

For example, consider the following fragment of a network file:

"layers" 3							
"neurons"	2	3					
"weights"	w_1,1	w_1,2	w_2,1	w_2,2	w_2,3	w_2,4	w_2,5	w_2,6

Here there are 3 layers; the input layer contains 1 neuron, the middle layer contains 2 neurons, and the top layer 3 neurons. The weights $w_{1,1}$ and $w_{1,2}$

connect the single input neuron to the first and second of the 2 second layer neurons respectively. The weights $w_{2,1} \rightarrow w_{2,3}$ connect the first neuron in the second layer with the neurons $1 \rightarrow 3$ in the top layer, etc.

The token "start time" marks the value of t that the network will start learning from, if the user desires continued teaching of the network. In other words, it is a measure of how long the network had been taught for if it was generated by a Ctrl-c or BREAK, or saved when it had finished learning or reached the "max sweeps" value of t.

The token "learn time" is equivalent to the "max sweeps" token and indicates to the software what value of t to stop training the network at if the user desires continued teaching of the network.

Command line options used to specify a network file and what is to be done with it (that is continued teaching or execution) are found in section 3.1.5. Command line options.

3.1.3. Teaching Data

The specifications for configuration and network files, and keyboard input contain references to "sample in" and "sample out" files. These refer to the teaching data, used by the program to teach the network.

The "sample in" file contains the data patterns or values desired for learning, and the "sample out" file contains the expected output for the sample input. Input/output pairs are matched; the first data items in the input file will produce the first data items in the output file after learning. The second set of input data will match the second set of output data after learning, and so on. Each input output pair is called a *case*.

3.1.3.1. Teaching Data File Format The start of an input case is signalled by a "[start]" token. There may be a user comment before the first token. Input data is matched to the input neurons one at a time, and in the order that they appear in the file. If there are more data items than there are neurons, the excess ones are thrown away up until the next "[start]" token occurs, and a warning is issued. The next case is then parsed. If for any case there are more input neurons than there is data, an error occurs.

The format for the file is as shown below:

α_1	β_1	γ_1	δ_1	...
α_2	β_2	γ_2	δ_2	...

3.1.3.2. Expected Data File Format Expected data files have the same format as teaching data files. The values in the expected data file are matched to output neurons in the same way as input data is matched to the input layer neurons. If

for any case there are more data items than there are neurons, the excess ones are thrown away up until the next "[start]" token occurs, and a warning is issued. If for any case there are more output neurons than there is output data, an error occurs. If there is not an identical number of input and output cases, an error occurs.

3.1.4. Test Data

Configuration and network files, and keyboard input, contain the tokens "execute in" and "execute out". The file names after these tokens refers to the files used as input and output for the network after it is trained. The "execute in" file contains data for the inputs of the bottom (layer 0) neurons, and the "execute out" file is used to store the output from the top (layer $n-1$) neurons for each case presented to the bottom layer. Any previous contents of the "execute out" file are overwritten by the new input cases' outputs.

3.1.4.1. Test Data File Format Format is identical to 3.1.3.1. Teaching data file format. Once again, if there is too much data the excess for each input cases is ignored (and a warning is issued), and if there is too little data an error occurs.

3.1.4.2. Output Data File Format The "execute out" file output format is similar to 3.1.3.2. Expected data file format. The only difference is that the length of the stored output line is modulated by the parameter following the token "output width", which is entered from the keyboard, or network or configuration files.

3.1.5. Command Line Options

To reduce keyboard input and increase the facility for repeated (automated) testing of program behaviour, all directives and flags used by the program can be set from the command line. Each flag or directive is selected by typing

$$\text{fishNET } \{-flag_0\}\{-\}\{flag_1\} \ldots \quad .$$

For example, to specify the flags $-x$ and $-v$ (the meaning of these two flags will be explained in detail below,) you could type fishNET $-x\ -v$ or fishNET $-xv$. The exception to this method are the $-c$ and $-n$ flags, which can only appear at the end of a "$-$" list or by themselves. This is because they refer to file names. For example,

$$\text{fishNET } -n\langle\text{netfile}\rangle\ -q.$$

A full explanation of each flag and directive appears below categorised by function.

3.1.5.1. Screen Input/Output Control $-v$ (verbose) flag

The $-v$ flag causes the program to produce verbose runtime information about the present status of the network. It displays the expected output from the network alongside the actual output, a running measure of errors, and the elapsed time, t. This process slows down execution speed by the amount of time taken for screen output.

This flag cannot be used with the $-q$ flag.

$-q$ (quiet) flag

The $-q$ flag causes suppression of all runtime messages. This flag is good for batch mode processing, and of course gives the fastest execution speed by virtue of minimal screen output.

This flag cannot be used with the $-v$ flag.

Default

If neither the $-q$ or $-v$ flags are set, the default mode is used. This mode causes fishNET to print rudimentary information about loading status, and displays the elapsed time and total error while the program is running.

3.1.5.2. Loading Configuration and Network Files from Disk $-c$ (load configuration file) directive

The $-c\{\text{path}\}\langle\text{filename}\rangle$ directive lets the user load a configuration file of parameters into the simulator. The format of the file is described in an earlier section entitled 3.1.1.2. Configuration file format.

This directive can't be used with the $-n$ directive.

$-n$ (load network file) directive

The $-n\{\text{path}\}\langle\text{filename}\rangle$ directive enables the user to load a pre-taught or manually created network file into the simulator. The format of the file is described in an earlier section, 3.1.2.1. Network file format.

This directive can't be used with the $-c$ directive.

Default

There is no default file loaded. If neither $-c$ or $-n$ is specified, the user is queried via the screen and keyboard for the appropriate parameter values.

3.1.5.3. Using the Network File $-e$ (execute) flag

The $-e$ flag is used after the $-n\langle\text{name}\rangle$ directive to tell the simulator to execute the network with the data files whose names are included in the network file. The network will *not* continue to be trained.

The $-e$ flag can't be used with the $-t$ flag.

$-t$ (teach) flag

The $-t$ flag is used after the $-n\langle$name$\rangle$ directive to tell the simulator to continue teaching the network with the sample input and output files whose names are included in the network file. When the network is "taught", execution with the data file (the name of which is also in the network file,) will occur.

The $-t$ flag can't be used with the $-e$ flag.

Default

If neither the $-e$ or $-t$ flags are used with the $-n$ directive, an error occurs. In other words, you must tell the program what you want to do with a network file!

3.1.5.4. Saving the Network File $-s$ (store "taught" network) flag

The $-s$ flag tells the program to store the network after it is taught. If this flag is set, once the errors drop below the threshold, the network is automatically stored in the file named through the keyboard, network, or configuration files. The format of the saved file is specified in an earlier section, 3.1.2.1. Network file format.

This flag can't be used with the $-e$ (execute) flag, or the $-d$ or $-x$ flags (see below).

$-d$ (don't store network) flag

The $-d$ flag indicates that once the network is taught, it will not be stored. If this flag is set, execution begins immediately after the network is taught, and although the network is not saved to disk, the output from the execution is.

This flag can't be used with the $-s$ or $-x$ save flags. If it is used with the $-e$ flag it is ignored.

$-x$ (store learning information and time) flag

The $-x$ flag is intended primarily for experimental use of the program. When this flag is set, the program will teach or continue to teach the network but when it is "taught" only the necessary parameters are saved to disk. These parameters are the weight modification factors α and ε, the number of layers and neurons in those layers, I/O files and time required to teach the network. This saves time and disk space for users of fishNET who are experimenting with the parameters' and data's effects on learning speed.

This flag can't be used with the $-s$ or $-d$ save flags, or the $-e$ execute flag.

Default

If none of the save flags is specified, the user is queried via the screen and keboard. The choices available are akin to the $-s$ and $-d$ flags (there is no equivalent $-x$ option).

3.1.5.5. Calculation of $\Delta w(t)$ The package fishNET offers two ways to calculate $\frac{\partial E}{\partial w}$ and $\Delta w(t)$, both enumerated in algorithmic form in the previous chapter.

-1 (each) mode

When the -1 flag is set, the program will calculate and apply $\Delta w(t)$ for each I/O case. This mode is only included for experimental purposes (for a description of the algorithm used, see the second algorithm in the previous chapter).

The -1 flag can't be used with the $-a$ flag.

$-a$ (all) mode

If the $-a$ flag is specified, the calculation of $\Delta w(t)$ is once per sweep (presentation) of all I/O cases. $\frac{\partial E}{\partial w}$ values are calculated for each I/O case and added together. $\Delta w(t)$ is calculated at the end of each sweep from the accumulated $\frac{\partial E}{\partial w}$ value. This is the most commonly used mode.

The $-a$ flag can't be used with the -1 flag.

Default

If neither the -1 or $-a$ flags are specified, the $-a$ mode is assumed. Unless $-q$ (quiet) has been set, a message is issued to this effect.

3.1.5.6. Printing the Help Message $-?$, $-h$ (help) flags

If the $-?$ or $-h$ flags are specified, the program doesn't run and a help message is displayed. The help message contains information on every flag and is located in the file `help.hlp` under all implementations of the program.

3.1.5.7. Unrecognised Options If any other option is specified, the program displays the message

"try typing 'fishNET $-?$' for help".

3.1.5.8. Default (no options set) If no options are set, the program asks for all information about file names and parameters, the same way it does if neither $-c$ or $-n$ is specified.

This is the recommended mode for "beginners".

3.2. Operations Performed

FishNET implements the back-propagation model of a neural network, as described in the previous chapter "The Back-Propagation Model". The shape of the network is flexible and specifiable by the user, and as many parameters as possible are easily modifiable.

The most important parameters are the network shape and size, and the initial values of α and ε. These values are user specifiable, and used during learning.

Input and sample output files are user specifiable and used for input/output pairs.

Final network operation output file is user specifiable.

Efficiency of operation is important, however the operations required are undertaken in such a way that flexibility of network size and portability of code are maintained.

3.3. Network Size

Any user specified network can have up to 8 layers. It can have any number of neurons in these layers, depending solely upon available memory of the particular hardware implementation.

Allocation of space for the network is completely dynamic.

3.4. File Input/Output Format

All files used by and created by the program are ASCII files. As a direct result, data files can be generated by hand with an editor, as can network and configuration files.

Chapter 4: Software Construction

In this chapter I will outline the construction of the code and the data structures used to implement the back-propagation algorithm. The package has been written entirely in C and all references to C "objects" such as keywords or variable names will be in `typewriter` style type. All developmental work was carried out on an IBM PC, using Microsoft QuickC^{TM}. The software has been sucessfully ported without modification to a Pyramid super-mini running Unix Version V. Execution speeds will be discussed in a later chapter, entitled "Performance".

This chapter is divided into two parts. The first part contains a description of the most important design decision, namely the data structures used for the dynamically allocated network. The second section details a rough outline of the flow of control of the software, especially as an aid to someone wishing to enhance the code. This section also details some important features of the code.

A printout of the C source code appears in toto in Appendix A. The source, include, object and executable files (as well as the TeX source for this document) are also on a diskette inside the back cover of this work.

4.1. Data Structure Design

As stated in the software specification, the code is to operate with networks of variable size and configuration, restricted only by available memory of the particular implementation. Because of this constraint, all allocation of space for the network (neurons and connections) is completely dynamic. The final result is a dynamically allocated structure that has no limits of size except that it must have fewer than 8 layers. This is not an unreasonable constraint, as even during testing no more than 4 layer networks were ever used, and some authors suggest that there need never be more than 4 layers for *any* problem.

In all sections following, the makeup of the individual C structures will be discussed, with field meanings, names and types given. The format followed will be — structure type name: field_0 description, type, name; field_1 description, type, name; etc. First I will describe the most important structure, namely that used for the network, then the other structures used for different purposes within the network.

4.1.1. Network Structure

A back-propagation network can be divided into 3 parts. A network is made up of layers. Each layer contains neurons. And each neuron has connections (weights) to the layer above. The most important constraint to the problem of a data structure for a network is the fact that it must be variable in size for each run (this also allows the software to deal with networks as large as available memory in any given system). Because of this the structure is made up of dynamically allocated arrays of pointers to pointers to structures.

4.1.1.1. Weights Weights are stored in a `struct` of type `WEIGHT`. Contained in `WEIGHT` are fields for: weight or connection strength, `double w`; $\frac{\partial E}{\partial w}$, `double dE_dw`; and $\Delta w(t-1)$, `double old_delta_w`.

4.1.1.2. Neurons Neurons are stored in a `struct` of type `NEURON`. `NEURON` contains fields for: output of the neuron, `double y`; a pointer to an array of weights for this neuron's connection to the layer above, `WEIGHT *w`.

4.1.1.3. Layer Layers are stored in a `struct` of type `LAYER`. A `LAYER` is made up of fields for: a pointer to an array of neurons for that level, `NEURON *neuron`.

4.1.1.4. Network A network is stored as a pointer to an array of type `LAYER` (that is, `LAYER *`). Throughout the code, this is how references to the network are passed.

4.1.2. Structures Used in Back-Propagation Calculations

Large numbers of intermediate variables are necessary during calculations using the back-propagation method. All are stored as dynamic arrays.

4.1.2.1. Temporary Values of $\frac{\partial E}{\partial x}$ Temporary values of $\frac{\partial E}{\partial x}$ are stored in an array of type `double`.

4.1.2.2. Temporary Values of $\frac{\partial E}{\partial y}$ Temporary values of $\frac{\partial E}{\partial y}$ are stored in an array of type `double`.

4.1.3. Input/Output Data Structures

To minimize relatively slow disk input/output for such a repetitive action as learning, all input/output data for learning and all input data for operation is read in at once when it is needed.

4.1.3.1. Neuron's Input/Output Value Every neuron's input value for layer 0 neurons and every neuron's expected output for top layer neurons is stored in `structs` of type `OP_DATA`. Contained in `OP_DATA` is a field `double x`, which is the value of input or output for the neuron in question. For each I/O case there is an array of `OP_DATA`.

4.1.3.2. Input/Output Cases Input and output cases are stored in a `struct` called `I_O`. This structure contains a pointer to a particular case, `OP_DATA *item`. All I/O data is stored in arrays of `I_O`, each element of which contains a pointer to a particular case.

4.1.4. General Data Structures

4.1.4.1. Data File Information Every data file used by fishNET has its information stored in a `struct` of type `DATA_FILE`. Contained in `DATA_FILE` are: the file name, `char name[35];` a single letter abbreviation of the file's type, `char type`; the number of input or output cases for this file, `int i_o_cases`; and the number of neurons in the layer this file is associated with, `int neurons`.

4.1.4.2. Most Commonly Used Parameters During the operation of fishNET, all constantly used data is stored in one `struct`, called `QUESTION`. Throughout the program, pointers to `QUESTION` (`QUESTION *`) are used to pass important data. `QUESTION` contains: the number of layers in the network, `int layers`; the number of neurons in every layer, `int per_layer[7];` the nominal output width, `int out_width`; the maximum number of learning sweeps, `int max_sweeps`; the learning parameter α, `double alpha`; the learning parameter ε, `double epsilon`; the name of the file where the network is to be saved, `char save_name[35];` a comment for the saved file, `char`

`save_comment[79]`; a chunk of information about network teaching and execution files, `DATA_FILE sample_in`; `DATA_FILE sample_out`; `DATA_FILE data_in`; `DATA_FILE data_out`.

4.1.4.3. Structure for Network Files When a network file is loaded, it contains the complete network (including weights), plus all the parameters necessary for operation. The function which reads in the network files returns a pointer to a struct called `LOAD_BOTH`. Contained in `LOAD_BOTH` are: a pointer to a network, `LAYER *network`; a pointer to a structure containing all run time information, `QUESTION *parameter`; and the value of t at which the network will start teaching, `int start_time`.

4.2. Program Design

There are two important functions which the program performs, and they are: training artificial neural networks, and running neural networks. The code is written so that both these functions are easily and possibly separately performed. Once the program has been given the necessary dimensions and parameters for the network, it will teach it. Given the network that you want executed, it will apply the required data to the input, calculate, then save the output. Hence, the program looks like:

```
expected data,  neural
configuration data  →  teacher  →  `operator'  →  output
↓  ↑
network file  run data
```

The items configuration and expected data, network file, run data, and output have been discussed in the previous chapter. Each of the other parts, the teacher and the neural 'engine' will be discussed below.

4.2.1. The Teacher

Teaching a network is broken into several phases. For each phase, the appropriate subroutine and its functions will be examined.

4.2.1.1. Reading the Parameters of Network Operation The action taken here depends upon the command line options set. If there are no options set, the program calls

```
parameter = input_parameters();
```

which returns a pointer to a structure containing the important parameters of network operation, read from the keyboard.

If the $-c$ option was used, the program instead calls

```
parameter = finput_parameters(config_file);
```

which returns a pointer to a structure, containing the important parameters of network operation, read from the file whose name is held in the variable `char *config_file`.

If the $-n$ option was used, fishNET loads the network at the same time as the operational parameters, so please see the next section on allocating space for the network.

4.2.1.2. Allocating Space for The Network If the network is to be trained from the start (that is, no network is pre-loaded), the program allocates space for it by

```
network = allocate(parameter);
```

This function allocates space (from the heap) and sets up random weights for a network of the desired size, and returns a pointer to it (`LAYER *`).

If the network is to be loaded from disk (the $-n$ command line directive was used), fishNET reads the parameters and allocates space for the network by

```
net_and_param = load_network_and_parameters(netfile);
```

The function takes as its arguments the name of the network file, and returns a pointer to a structure which contains: a pointer to a structure which holds the parameters; a pointer to a network, with the values read from the file as the weights; and an integer value of the time t the network is to start learning at. For more information about the data structures used see 4.1. Data structure design.

4.2.1.3. Loading the Expected Data Once the network has been allocated, the data is read in and stored in the structure previously described in 4.1.3. Input/Output data structures. The function call returns a pointer to an array of type `I_O`, that is `I_O *`. The call is

```
 input= get_data(parameter->sample_in);
output= get_data(parameter->sample_out);
```

4.2.1.4. Teaching the Network The network is taught by the function

```
learn(network, parameter, start_time);
```

This function uses the information in the variable `parameter`, which is actually a pointer to a structure of type `QUESTION`, to modify the network, which is pointed to by `LAYER *network`. The initial value of t used by the routine is `int start_time`. The algorithms are explained in Chapter 2, and are implemented exactly as documented there.

If the first algorithm is being used, ($-a$ specified, or the command line default,) `learn` calls the routines `operate`, `back_propagate`, and `apply_delta_w` to implement the algorithm. Actually, `apply_delta_w` is executed first, because a `do ...while` loop is used to test the number of errors, which is not known until the routine `back_propagate` is called. Hence to make sure that `apply_delta_w` is not called once too often, (ie when the error is actually zero,) it is run first. This works because as the network is allocated by fishNET, it is initialised and so the first running of `apply_delta_w` does nothing.

The routine `operate` is detailed in the next part, The neural 'engine'.

`Back_propagate` calculates the error $\frac{\partial E}{\partial w}$ for each case and adds it to the total kept in memory for every weight. The error is calculated by comparing the output with the expected output, and is described earlier.

`Apply_delta_w` is used to calculate and then add $\Delta w(t)$ for every weight in the network.

If the second algorithm is used (the -1 flag was used), the functions called are slightly different. First, `learn` calls `operate`, then a function called `back_propagate_apply_delta_w`. This routine is a combination of the two like–named routines described above, `back_propagate` and `apply_delta_w`. It operates by calculating $\frac{\partial E}{\partial w}$ for every case and using this to calculate $\Delta w(t)$ and apply it for every case. This appears to make the network converge much more slowly, but is included for experimental purposes.

4.2.1.5. Saving the Network Once the network is fully 'taught', or the maximum value of t is reached, the action depends upon the setting of the command line options $-d$, $-s$, and $-x$.

If $-d$ was specified, execution occurs immediately and the network is discarded.

If $-s$ was selected, the network is saved to disk by a call to the function

```
store_network(network, parameter);
```

and execution occurs immediately.

If $-x$ was specified, the learning parameters are saved by

```
store_learnt_parameters(parameter);
```

and the program exits without execution.

4.2.2. The Neural 'Engine'

4.2.2.1. Operating With a Network The network passed to the function has the input applied to the bottom layer and the output is stored in the top (output)

layer of the network. The input of every neuron is the sum of the outputs of all the neurons below times their interconnecting weights. The output of every neuron is its input times the transfer function. The call that performs these functions is

```
operate(network, input_case->item, parameter);
```

Chapter 5: Performance

The performance of software models of neural networks is probably their biggest drawback — it is a classic tradeoff between flexibility and speed. A hardware model is extremely inflexible, limited in size, and very cumbersome, yet it offers performance that doesn't degrade as the network gets bigger. Software models, on the other hand, are extremely flexible but horrendously slow in *all* implementations. Here we examine just how slow a software simulation is.

An analysis of both execution and learning time, and the amount of memory required for any network is given in the following brief chapter. The aim is to give the reader some simple rules of thumb for calculating: how long a simulation using fishNET will take in any particular implementation, and how big that simulation can be.

Problems with performance of fishNET tend to take the form of execution speed, not lack of memory. Long before available memory with a PC runs short, patience inevitably does. Lack of memory has actually only happened once, and this was a deliberate experiment to see how large a network could be. In reality, the time taken to learn (in physical seconds, not internal learning time t) becomes so large that it is nearly pointless continuing. Just how large is feasible? Hopefully with the help of this chapter the reader can calculate how long it will take and maybe rethink network size and shape. Thus it is execution speed, not memory size, which is the most important constraint in PC implementations.

5.1. Hardware

A description of all the hardware configurations that fishNET was tested with follows:

5.1.1. The (almost) Standard IBM-PC

As mentioned above, fishNET's development was undertaken entirely on a 1985 vintage true-blue IBM-PC. The 4.77MHz intel 8088 has been replaced by a National V20 workalike processor, which is marginally faster with screen and disk I/O and up to 5 times faster for some processor operations. The speed improvement, however, is only about 20% in general. The machine has 640k of RAM.

5.1.2. The Turbo-Charging 8087

After finding that learning times for small networks was several hours, the IBM-PC above was fitted with a genuine intel 8087. The speed improvement was in the order of a factor of 12.

5.1.3. The Pyramid Super-Mini

This machine literally turned days into minutes. It is a Pyramid model 9810, which has 16 Mbytes of actual and 200 Mbytes of virtual memory with a RISC architecture, so fishNET simulations of painful size for the PC were not much of a burden. Really.

5.2. Learning and Execution Speed

To calculate learning and execution speed is easy on a machine as slow as a PC, since it is possible to simply use a stop-watch and observe the appropriate screen output. On larger multi-tasking systems with buffered screen output this is not possible. Hence the timings shown for the Pyramid err on the high side (the machine actually runs faster). This is because it was necessary to use operating system information on execution time which involved the complete running of the program from invocation to termination, not just the different phases such as learing and execution. However, the case chosen for calculations had a reasonably large number of learning iterations, so the result is accurate to within a factor of 1.5.

5.2.1. Learning Speed

For simplicity's sake, consider the application of the back-propagation algorithm to a weight as a single operation. Then, it is possible to calculate the learning speed by measuring the length of time taken for a single application of the back-propagation algorithm and the application of $\Delta w(t)$, then dividing by the number of I/O cases and dividing by the number of weights. (For more accuracy, several dozen sweeps were timed and the result divided by the number of sweeps.)

This technique gave the following results:

Implementation	Length of time for a 'single' b.p. weight calculation
PC_V20 (8088)	9.17 msec
PC_8087	733 μsec (=0.733msec)
Pyramid	20 μsec (=0.02msec)

A good approximation of speeds in general then is

$$\text{PC}_{\text{V20 (8088)}} = \frac{\text{PC}_{8087}}{12.5} = \frac{\text{Pyramid}}{459}$$

5.2.2. Execution Speed

Calculation of execution speed is a more difficult task. Firstly, as after each execution there is a lot of disk output, the stop-watch time for the PC will be grossly distorted (floppy disk systems are even worse). Hence the major mathematical operations involved will be analysed, with their timings based upon reference [27]. Secondly, disk output time will be an order of magnitude slower than processing speed in most implementations. No attempt to analyse this time is made here, as it is too application dependent.

For the simulator to operate upon a network (that is, use the runtime data), there are a number of mathematical operations involved. A summary is

1. $1\times$ transfer function calculation per neuron, which is
    1. $1\times$ negate,
    2. $1\times$ `exp` calculation,
    3. $1\times$ `double` add, and
    4. $1\times$ `double` divide.
2. $1\times$ `double` multiplication per weight, and
3. $1\times$ `double` addition per weight.

Hence, using [27], we find that the approximate execution speed for a single input case is of the order of

$$\begin{aligned}
\text{PC}_{\text{V20 (8088)}} &= (n_{\text{neurons}} \times 41.61 \times 10^{-3}) \\
&\quad + (n_{\text{weights}} \times (0.753 \times 10^{-3} + 2.71 \times 10^{-3})) \text{ sec} \\
&= (n_{\text{neurons}} \times 41.61) + (n_{\text{weights}} \times 3.463) \text{ msec} \\
\text{PC}_{8087} &= (n_{\text{neurons}} \times 1.08 \times 10^{-3}) \\
&\quad + (n_{\text{weights}} \times (0.186 \times 10^{-3} + 0.187 \times 10^{-3})) \text{ sec} \\
&= (n_{\text{neurons}} \times 1.08) + (n_{\text{weights}} \times 0.373) \text{ msec}
\end{aligned}$$

These values are very approximate. However, they still show an interesting trend. Assuming that the Pyramid is at least 37 times faster than the PC_{8087}, we find

$$\text{Pyramid} = (n_{\text{neurons}} \times 29.2) + (n_{\text{weights}} \times 10.1)\ \mu\text{sec}$$

So for a typical problem with 3 layers, (195 neurons in the input layer, 30 in the middle layer, and 5 in the output layer, or 6000 weights in total,) the expected execution times are

$$\begin{aligned} \text{PC}_{\text{V20 (8088)}} &= 30.35 \text{ sec} \\ \text{PC}_{\text{8087}} &= 2.49 \text{ sec} \\ \text{Pyramid} &= 67 \text{ msec} \end{aligned}$$

These are close to the roughly measured times.

As you can see, the fact that there are so many more weights than neurons means that calculations involving weights consume the most significant part of the time, even though the execution speed of a neuron is typically $3 \rightarrow 12$ times slower. In these calculations, memory addressing time has been ignored, as it is $100 \rightarrow 1000$ times faster than floating point operations.

5.3. Memory Used

Once again, we are only searching for a rule of thumb: on single processor systems such as PCs, memory resident device drivers and RAM disks can make it difficult to accurately guage free memory (the MS–DOS utility `chkdsk` gives this information, as well as disk free data), and on multi-tasking systems, reasonably sized simulations aren't going to pose much of a problem.

A quick calculation follows. Memory used is broken into four categories:

1. a structure of type `QUESTION`,
2. the network itself,
3. a variable amount of space for teaching and execute data, and
4. a variable amount of space for calculation purposes.

The space taken up by the structure of type `QUESTION` is intentionally always the same. The actual number of bytes varies slightly from implementation to implementation (as the size of an `int` varies). For a PC application

$$\begin{aligned} \texttt{QUESTION}_{\text{space}} &= 4 \times (36 \times \texttt{char} + 2 \times \texttt{int}) + (10 \times \texttt{int}) \\ &\quad + (2 \times \texttt{double}) + (114 \times \texttt{char}) \\ &= (258 \times \texttt{char}) + (2 \times \texttt{double}) + (18 \times \texttt{int}) \\ &= (258 \times 1) + (2 \times 8) + (18 \times 2) \\ &= 310 \text{ bytes.} \end{aligned}$$

It will become obvious that this is hardly significant.

The network requires space for

1. `LAYER` structures,

2. `NEURON` structures, and
3. `WEIGHT` structures.

Hence the amount of space required is

$$\begin{aligned}
\texttt{LAYER}_{\text{space}} &= (\texttt{NEURON *}) = 4 \text{ bytes}, \\
\texttt{NEURON}_{\text{space}} &= (\texttt{WEIGHT *}) + \texttt{double} = 12 \text{ bytes}, \\
\texttt{WEIGHT}_{\text{space}} &= 3 \times \texttt{double} = 24 \text{ bytes}.
\end{aligned}$$

So the equation for memory usage by a network becomes

$$\texttt{NETWORK}_{\text{space}} = (n_{\text{layer}} \times 4) + (n_{\text{neuron}} \times 12) + (n_{\text{weight}} \times 24),$$

where n_{layer} is the number or layers, etc.

Memory required for teaching and execution data depends upon the number of neurons in the input and output layers, and the number of I/O cases. Every neuron on the input has a piece of data for learning and execution for each case. Every neuron on the output has a piece of data for comparison during learning for each I/O case. So

$$\begin{aligned}
\text{data}_{\text{space}} &= (n_{\text{layer 0 neurons}} \times (\text{learning cases} + \text{execution cases}) \times \texttt{double}) \\
&\quad + (n_{\text{output layer neurons}} \times \text{learning cases} \times \texttt{double}) \\
&= (n_{\text{layer 0 neurons}} \times (\text{learning cases} + \text{execution cases}) \times 8) \\
&\quad + (n_{\text{output layer neurons}} \times \text{learning cases} \times 8).
\end{aligned}$$

The next facet to consider is space used during calculations. Arrays of type `double` are used during back-propagation calculations (in the functions `back_propagate` and `back_propagate_apply_delta_w` as described in the previous chapter) to store values of $\frac{\partial E}{\partial x}$ and $\frac{\partial E}{\partial y}$. The arrays refer to different layers of neurons at a time, but for the rule of thumb we will invent a worst case. The worst case is where we have two big layers of the same size next to each other, so

$$\text{calculation}_{\text{space}} = (n_{\text{neuronsin biggest layer}} \times 16)$$

Putting all these parts together,

$$
\begin{aligned}
\text{Memory required} &= \texttt{QUESTION}_{\text{space}} + \texttt{NETWORK}_{\text{space}} + \text{data}_{\text{space}} + \text{calculation}_{\text{space}} \\
&= 310 + (n_{\text{layer}} \times 4) + (n_{\text{neuron}} \times 12) + (n_{\text{weight}} \times 24) \\
&\quad + (n_{\text{layer 0 neurons}} \times (\text{learning cases} + \text{execution cases}) \times 8) \\
&\quad + (n_{\text{output layer neurons}} \times \text{learning cases} \times 8) \\
&\quad + (n_{\text{neuronsin biggest layer}} \times 16) \text{ bytes.}
\end{aligned}
$$

Using again the example of a 3 layer (195, 30, 5) network, (with 5 learning cases and 20 execution cases,) we get

$$
\begin{aligned}
\text{Memory required} &= 310 + (3 \times 4) + (230 \times 12) + (6000 \times 24) \\
&\quad + (195 \times 25 \times 8) + (5 \times 20 \times 8) + (195 \times 16) \\
&= 190\,002 \text{ bytes} \\
&= 185.5 \text{ kbytes.}
\end{aligned}
$$

5.4. An Interesting Comparison — Software versus Wetware

The human brain has been dubbed 'wetware' by some authors. How does wetware compare with a single processor software simulation of the brain? Consider a very, very simple example.

According to Feldman et. al. [12], the "execution" time for a human neuron to produce an output from its inputs is in the order of 5 milliseconds. However, *all* neurons work at the same time (within a layer, say) so a 3 layer network should produce an output in 15 milliseconds. Considering the gross simplifications here, let us increase this estimate by an order of magnitude, and say that for a simple perceptual problem, wetware produces an answer in 200 milliseconds. Assume we used a tiny fraction of the brain, say 300 000 neurons. Assume again that we have only 5 000 connections per neuron. Then if the 300 000 are equally divided in 3 layers, we have 1×10^9 weights. And it can produce an answer in 200 milliseconds.

A Pyramid would require $\sim 7.7 \times 10^3$ seconds $=$ 2 hours 9 minutes to do the same calculation, and a plain PC would take $\sim$ 41 days 4 hours. This also involves *no* consideration for how long it would take to teach the software models by back-propagation.

Software artificial neural networks will never be able to perform a meaningful portion of the human brain's functions, no matter how fast the machine, if the only available technology is a few planar silicon processors. True multidimensional parallel processing such as optical or biological processors offer the only possible path. Or we will be stuck with simulations of the same order as the ones investigated in this thesis. Simulations of this size are however remarkably useful, as the next part of this thesis shows.

Part 3: Using FishNET

Chapter 6: Introductory Experiments

The back-propagation algorithm described in an earlier section was applied to several problems using the software package fishNET. Various input data was used, and the number of middle (intermediate) layer neurons was varied. The learning parameters α and ε were also modified.

Due to most of the work being carried out on a PC, there was an upper limit to the number of neurons used in simulations. The case dealt with in detail was extensively tested with parameter variations. The other examples here will be examined briefly to show how versatile a neural network is.

The intent of this chapter is to illustrate the initial experiments that were performed with fishNET to prove that it worked as desired. It is possible to use fishNET to do many different tasks simply by creating the appropriate input and output data. The software will then generate a generalised network, with no other human input. It is a remarkable tool.

Later chapters investigate further the effects of parameters on network behaviour, and the effect of casualties within the network.

6.1. The First Test — O and X Recogniser

The first piece of test data that was created and run on the network was a simple O and X recogniser. There were 9 neurons (bits) on the input, $6 \rightarrow 9$ neurons in the middle layer, and 2 output neurons (bits), each one of which signalled either a O or a X. The network converged to a solution in about 100 learning iterations for 7 intermediate layer neurons; the speed of learning versus number of intermediate neurons will be discussed later.

The input and output data is shown below

Input 1		
1	0	1
0	1	0
1	0	1

Input 2		
1	1	1
1	0	1
1	1	1

Output 1	
0	1

Output 2	
1	0

Figure 8. Two simple input/output case pairs.

To test if the network generalised, even for such a simple case, input data not used for training was input (such as 0.5s being used to replace some of the 1s in the shapes). The network produced the correct results.

6.2. A More Ambitious Test — Shadow Encoding

Burr [25] discusses the use of a 13 segment shadow encoder as a means of describing hand written data for input to a neural network. He aimed to be able to recognise the alphabet as written by a particular writer. He took 8 samples of the alphabet written over a period of time, and then trained the network on 4 of the samples, using the other 4 as test data to see if the network could successfully recognise them. His approach was remarkably fruitful, with a recognition rate of up to 99%. The example given here was not that large.

Shadow encoding can be illustrated by figure 9, from Burr [25].

Figure 9: Shadow encoding of the letter 'S'

Several attempts were made at using this shadow encoded alphabet with fishNET. Initially, 5 letters were encoded and the network was trained to recognise them within a hundred or so learning iterations. However, when the 5 letters were replaced by the full alphabet, the computational time per learning iteration of the network rose by a factor of more than 7 (since 5 times as many input/output cases, plus more neurons in the middle layer). Hence this network was never fully allowed to converge to a solution. Possible methods of speeding convergence are dealt with in a later chapter.

6.3. Dot Matrix Letter Encoding

This was the other major initial test experiment. Shadow encoding, although partly removing some of the inherent problems of neural network character recognition such as rotation, can't be easily visualised. The aim of this thesis is to provide qualitative judgements on network behaviour, so for this reason a more easily visualised encoding technique was sought. Dot matrix encoding of several letters of the alphabet was chosen.

Various sizes (geometries) of the input layer were tried, and to make symmetric letters easier to encode an odd number of neurons was always used in both dimensions. The number of neurons in the bottom layer was increased as the processing speed increased due to the addition of the floating point processor (8087). First attempts were for a 5×7 grid of input neurons, but this was too small for reasonable test data to be generated. 9×11 was also tried, before finally selecting 13×15. This rather large number of neurons, (195 in the bottom layer,) gives a dot matrix encoded letter that is of reasonable quality, while still being easily computable in a network on a PC with an 8087.

This example is extensively discussed in the next chapter, and is used throughout the thesis for experimental purposes.

Chapter 7: Experiments on learning

In this chapter, the dot matrix encoding of English language characters will be considered in detail. There are two important facets to consider in these experiments: learning time, and error rate. Learning time is the length of time in machine (or algorithm) time units t which is required for the network to produce results which are sufficiently accurate. Error rate is calculated as the total error, or E, which is the sum of the square of the difference between the expected output and actual output of the network.

Two variables tend to govern both the learning time and the error rate. These variables are the learning calculation parameter ε and the number of neurons in the middle layer. Both are analysed in detail below.

For the rest of this thesis, we will consider only 3 layer networks. In neural network problems, the fourth layer is usually used to remove rotational and translational problems with pattern recognition: as the behaviour of this 3 layer network will be examined for behaviour with rotated and other data, no fourth layer will be considered. As well, the error threshold level was fixed at 0.2. That is, a neuron which is supposed to have an output of 1 or 0 is not flagged as being in error if its output is greater than or equal to 0.8, or less than or equal to 0.2, respectively. Some authors drive their networks to 0.9 and 0.1, but the extra computational effort for a PC implementation would be considerable (in many cases, most neurons were within 0.1 of their desired value, with only a few being as far as 0.2 away). The final invariant used here is the spread of the initial weights. They were each randomly set to a value between $+0.3$ and -0.3.

The two learning calculation variables, α and ε, can have a wide range of values. Just *how* wide appears to depend largely upon the problem. For all experiments examined here, α was set to 0.9. This doesn't pose a problem, because beyond values of a few tens of t the equation $\alpha e^{-t} \Delta w(t-1)$ drops to almost zero. Hence α is not really a significant factor in the calculations beyond the very beginning, and most authors set $\alpha = 0.9$.

So all learning equation parameter variations refer to ε. The number of neurons in the second layer (referred to as layer 1, since counting starts at 0,) was also widely varied during experimentation. We examine below the effects of these parameters' variations on learning time t and error rate E. A summary is included at the end of this chapter.

7.1. Variation of Learning Time with Parameters

This experiment dealt with the length of time taken to fully train the network with the error criteria set above. However, there is a subtle difference to most authors approach to this problem.

As initial weights varied randomly from $+0.3$ to -0.3, most experiments here were run 4 times in order to get more statistical accuracy. Just under 80 simulations were performed, and the averages and standard deviations calculated. Both parameters of interest, ε and neurons in the middle layer, were varied. ε values were set at 0.3, 0.5, 0.9, and 1.0. Configurations of layer 1 neurons that were used are 15, 20, 35, 50, and 70. Up to 4 tests were carried out with each one.

The results are tabulated and graphed below. Where possible, (and it as in all but one case due to a lack of data,) means and standard deviations (SD) are shown.

neurons	ε							
in	0.3	0.5	0.9	1.0				
layer 1	Mean	SD	Mean	SD	Mean	SD	Mean	SD
15	416	33	204	25	217	50	226	–
20	348	24	170	3	214	62	202	32
35	278	12	160	8	195	23	254	60
50	235	10	158	2	288	46	435	172
70	220	15	190	14	337	46	654	210

Figure 10. Table of mean learning times and standard deviations.

Figure 11: Mean learning time versus number of neurons

From the graph and table in figures 10 and 11, some trends become clear: an increase in ε does not necessarily make the network converge to a solution any faster. In fact, as ε increases beyond about 0.5, the network begins to become unstable during learning.

This phenomenon can be observed with the $-v$ option set in fishNET. With large ε, some output neurons converge quickly towards the desired result. However, the others tend to move in the opposite direction to the one desired. After several tens of t, the outputs suddenly begin to "swap", and the neurons that had an output very close to the desired output (some will be less than 0.05

away) begin to move away from the desired result, and the outputs which were the opposite tend to moves towards the correct output. This see-saw effect can continue for several hundred t, until usually it will decrease in oscillation size, and all outputs will move slowly toward the desired result.

Before the final set of experiments quoted here were run, a previous set of ε values was tried. In this group were the values 1.2, 1.5, and 2.0. None of these values converged systematically in less than $t = 5000$, which is the default limiting value. This fact, and the standard deviations of the learning times examined below, tends to indicate that $\varepsilon = 1.0$ is near the boundary of stable learning for this particular problem.

The oscillatory effect also seems to get worse with more neurons. The network appears to have trouble learning for both a large ε and a large number of intermediate layer neurons, and the huge standard deviations in these cases point this out. In fact, for the case where $\varepsilon = 1.0$, and middle layer neurons numbered 70, the learning time varied from between $t = 361$ and $t = 838$. This is a very different result to the $\varepsilon = 0.3$ or $\varepsilon = 0.5$ standard deviations. A graph of standard deviation versus neurons in layer 1 for all ε values is shown in figure 12. In general, for $\varepsilon > 0.5$, the standard deviation of learning time (and hence the instability during learning) increases as a function of both ε and neurons in the middle layer.

Figure 12: Standard deviation versus number of neurons

The effect of ε and neurons in layer 1 on error rate will be examined in the next section.

7.2. Variation of Error Rate with Parameters

The experiment detailed here entailed taking a random sample of each of the ε and middle layer values used in the previous experiment and calculating the total error for a constant set of input data. Hence there are 20 values of total error.

The input data used is shown in the appendices. It involves 4 sets of data for each of the 5 letters, including noisy and distorted shapes. None of the data used for testing had been used to train the network, so the experiment was a good one to test the network's level of generalisation. Due to the large number of computations necessary, and the disk space needed to store the networks, (training was done on the Pyramid 9810, but all analysis, that is execution, was done on the PC_{8087},) only one of each category was tested. This means that the shape of the graph of total error versus neurons should be viewed only for trends and may not itself be overly statistically accurate. The data appears below:

neurons	ε			
(layer 1)	0.3	0.5	0.9	1.0
15	6.15	6.17	5.61	4.97
20	6.03	6.01	5.57	5.83
35	6.17	6.30	5.46	5.54
50	6.11	6.05	5.80	5.34
70	6.25	6.41	5.81	6.10

Figure 13. Table of total errors.

Figure 14: Total error versus neurons

Figure 14 shows a graph of the data. Interesting details can be drawn from it. Firstly, the total error doesn't decrease as the number of neurons in the intermediate layer grows larger. Some authors suggest that the network fails to generalise when the number of neurons grows too large. The data sample here is too small to confirm or deny this, but there seems to be a slight decrease, then increase in total error, from left to right.

More interesting still is the effect of ε upon the network. Due to computational constraints, only a reasonably small data sample was used, but it still appears that the error is a maximum for some value of ε between about 0.3 and 0.5, and probably gets smaller as ε gets smaller than 0.5 and also smaller as ε gets larger than 0.5. (Note again that the network would not reliably converge for values of ε above 1.0.)

Comparing the graphs in figures 11 and 14 yields even more important results. Networks that take longer to teach (converge) produce a smaller error than those which converge quickly to a solution. An explanation for this follows. Faster converging solutions from the back-propagation algorithm are more likely to settle in non-optimal minima in the solution space. This is because the network will be "coarser" due to the obviously larger values of $\Delta w(t)$ that were added during back-propagation calculations. Because these values were larger, the network would probably not be as finely tuned as one which took longer to converge from smaller $\Delta w(t)$s. (Another way of thinking of this is to try to imagine that the required solution is some symmetrical pattern of weights. If the values of $\Delta w(t)$ are bigger, the network may satisfy the error criteria but still not be particularly symmetric.) Hence slower trained networks are more likely to accurately generalise the problem, and produce fewer errors.

One final point to explain is why total error was used in this experiment instead of actual error. Even though none of the test data had been seen by the network, the number of actual errors (that is incorrectly recognised letters) *was always the same for all cases above*, and was a very small portion of the data. (One can tell the selected pattern, even though the output for the selected pattern's appropriate neuron is much less than the desired 0.8, by looking for the most active neuron of the 5 output neurons.) In fact only the most distorted "letters" generated

actual errors (see the appendices), so total error was used to get some spread between the various cases.

The next experiments examine the effect of casualties in the network upon its performance.

7.3. Summary

The results presented in this chapter highlight some features of learning with this problem. There can be large variations in the learning time for a particular problem, depending upon the values of ε and the number of middle layer neurons. In general, learning time is shortest for values of ε of between 0.3 and 0.5. ε values within this range cause networks to learn faster as the number of neurons in the middle layer increases. However, for larger values of ε (of between 0.9 and 1.0), the learning time *increases* as the number of neurons gets bigger. A very large variance in learning times was observed for these values. This was especially true for $\varepsilon = 1.0$, which to be on the outer limits of convergence for large numbers of layer 1 neurons.

From looking at the error rate, we can make statements on the effect of learning time on errors. The error is slightly smaller for networks that take longer to teach, and an explanation for this has been proposed. There is not a dramatic difference in error rate between the different ε and neuron values (about 20% variation at most).

Chapter 8: Effects of casualties

One feature that separates neural networks from other computational techniques is the fact that knowledge is distributed amongst the weights in the network. Hence, any network is redundant, and can sustain a certain amount of injury without being overly affected. Once again, just how much damage a neural network can take is something that has not yet been widely investigated, but the simple experiments detailed here, even though possibly not statistically sound, (only one network was used,) provide an important insight into general network behaviour.

The approach taken involved selection of a network, then damaging the network and examining its behaviour for the same data set used in the last chapter. The network selected had the largest ε used, $\varepsilon = 1.0$, and the smallest number of middle layer neurons, $n_{\text{layer 1}} = 15$. This combination was chosen because it had the lowest error rate out of any of the networks examined previously, and because it was easiest to edit, with only a small number of middle layer neurons.

Casualties in the network took two forms. First, random weights were damaged (set to zero) to simulate the destruction of connections between neurons. Various percentages of total connections were removed, namely 2.5%, 5%, 10%,

20%, 30%, and finally 40%. The total error, (as before, E, the sum of the square of the difference between actual and expected outputs,) and actual error (the number of cases where the output layer neurons chose the wrong pattern,) were calculated.

Second, individual neurons in the middle layer were "damaged," that is all the connections between their outputs and the top (output) layer were set to zero. The effect of the removal of between 1 and 7 out of 15 possible neurons was examined, with total and actual error again being used.

We examine below these two types of casualties. A summary of results is given at the end of this chapter.

8.1. Casualties in Connections

FishNET, the neural network simulator package used in this thesis, stores networks as ASCII files. Hence, generating casualties in the network is a simple matter of randomly picking a weight and setting it to zero. This procedure was followed several hundred times, to produce weight casualty rates of 0%, 2.5%, 5%, 10%, 20%, 30%, and 40%. The results of this damage appear in the table below. Actual and total error are given for each percentage group. | | Casualty Rate (weights) | | | | | | | — | — | — | — | — | — | — | — | | Error | 0% | 2.5% | 5% | 10% | 20% | 30% | 40% | | Total | 4.97 | 5.11 | 6.09 | 6.60 | 6.82 | 7.97 | 16.23 | | Actual | 3 | 3 | 4 | 4 | 3 | 4 | 15 |

Figure 15. Table of total and actual errors.

These figures are graphed in figure 16.

Figure 16: Total and actual error versus % weight casualties

Some very interesting features emerge from this data. With 2.5% casualties, there is no appreciable difference in actual or total error. Once the number of casualties is doubled to 5%, the total error increases by 20% and the actual error rate by 25% (but as we shall see, this is not an overly relevant figure). However, for a further doubling of casualties to 10%, there is only an 8% increase in total error, and doubling the damage again causes a *decrease* in actual error, indicating that there is some random variation in actual error results (so the previous increasing figure can be neglected). From 0% to 20%, there is only a 37% increase in total error, and *no* increase in actual error. So it appears that network behaviour is not greatly modified at casualty rates of up to 20%. By the time casualties reach 30%, total and actual error are rapidly on the increase.

Looking at the graph, we see that the error starts to increase rapidly beyond casualty rates around 30%. Error rates of both kinds become exponentially worse, and the network can be said to be no longer able to recognize patterns meaningfully. (Actual error has risen from $\frac{3}{20}$ or $\frac{4}{20}$ cases at $20 \rightarrow 30\%$ casualties, to $\frac{15}{20}$ by 40% casualties.)

From this data, we draw the conclusion that our $\varepsilon = 1.0$ and $n_{\text{layer 1}} = 15$ network can sustain random injuries in up to between 20% and 30% of weights, and still function satisfactorily. This is quite incredible.

Larger networks (with larger intermediate layers) probably behave in much the same way. Of course, for the same *number* of weight casualties the effects will be less, but for the same *percentage* of the total weight population the behaviour will probably be the same.

8.2. Casualties in Middle Layer Neurons

In this experiment, various numbers of middle layer neurons were disabled, by setting the connecting weights between these neurons and the output layer to zero. This simulates the destruction of the individual processing elements. Here the effect of the removal of between 0 and 7 out of 15 middle layer neurons is examined.

A table containing total and actual error data for a varying number of neurons removed is shown in figure 17. | | Number of Casualties (neurons) | | | | | | | | — | — | — | — | — | — | — | — | | Error | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | | Total | 4.97 | 4.97 | 5.27 | 7.47 | 8.39 | 9.76 | 10.44 | 11.21 | | Actual | 3 | 3 | 3 | 6 | 6 | 9 | 10 | 7 |

Figure 17. Table of total and actual errors.

These figures are graphed in figure 18.

Figure 18: Total and actual errors versus number of layer 1 neuron casualties

Another lot of interesting results comes from the table and graph. With up to 2 neurons removed, there is no appreciable increase in actual or total error. With more than 2 removed, the actual error doubles and continues to increase as more neurons are removed. As the graph shows, the network rapidly decreases in usefulness as the number of damaged neurons increases.

The data, then, shows us that the removal of a neuron has a more dramatic effect as a straight percentage of neurons in the middle layer than weight damage has. If more than about 13% ($\frac{2}{15}$) are damaged the network can't really be used. It is possible as well to look at the removal of neurons as a form of weight damage. By looking at it in this way, some remarkable conclusions can be drawn about the representation of knowledge in the network, without even having to examine the distribution of weight patterns in the network.

If one neuron out of 15 is damaged, then this is equivalent to the systematic (instead of random) removal of $\frac{1}{15}$th of the total neurons in the network. Once the number of systematically removed weights reaches more than 13%, the performance degrades rapidly. This fact reveals some important aspects of network behaviour. Even though the percentage of weights damaged systematically is lower, the effect is much greater than random weight damage.

Behaviour such as this indicates that although the knowledge of any pattern is widely distributed among the weights in the network, the selection of features is centralised into groups of neurons in the middle layer. Hence the injuring of intermediate layer neurons removes almost entirely, instead of partially as for weights, a network's ability to recognise features of a pattern, which in turn affects the output of pattern recognition by the top layer.

For illustrative purposes, a mean time between failure analysis is undertaken below to show just how reliable a neural network is with regard to failure in intermediate layer neurons.

8.3. Mean Time Between Failure Analysis

Angus [28] gives a simple calculation for mean time between failure (MTBF), which we use below for illustrative purposes. From [28],

$$\theta_{\text{sys}} = \frac{\sum_{i=0}^{n-k} \binom{n}{i} (\lambda/\mu)^i}{k\lambda \binom{n}{k} (\lambda/\mu)^{(n-k)}},$$

where the notation is explained as

n	number of nominally identical units
k	minimum number of operating units for system to operate successfully
θ_sys	mean (successful operating) time between system failures
τ	mean time to repair an individual unit
θ	mean time to failure for an individual unit
λ	failure rate for an individual unit, $\lambda 1/\theta$
μ	repair rate for an individual unit, $\mu 1/\tau$

and

$$\binom{n}{i} = \frac{n!}{i!\,(n-i)!}.$$

Imagine that a neural network is made up of individual, replaceable elements in a hardware simulation, with a structure identical to the one used in this and

the previous chapter (3 layers, 195, 15, and 5 neurons). So 2 out of the 15 neurons in the middle layer can be damaged and have almost no effect on network behaviour. Assume that we have some method of knowing that any individual neuron (op-amp) has malfunctioned and can replace it within 1 hour. Also assume that the mean time to failure for each op-amp is $10\,000$ hours of continuous operation. Then, MTBF (θ_{sys}) for this network is 7.337004×10^8 hours, or $83\,756$ years. Quite a long time. (These figures come from $n = 15$, $k = 13$, $\lambda = \frac{1}{10\,000}$, and $\mu = \frac{1}{1}$.)

8.4. Summary

This chapter's results reveal some important facts about the representation of knowledge in back-propagation neural networks, as well as showing just how fault tolerant neural networks are.

Random removal of weights has shown that the network is quite immune to failures of this kind. In fact, between 20% and 30% of weights can be set to zero, with very little effect upon the operation of the network. This implies that the knowledge of all patterns learnt is spread over all the weights in the network.

Removal of middle layer neurons (or systematic removal of weights) reveals a more astounding fact. It is widely recognised that neurons in the middle layer act as feature extractors, reacting to particular parts of an input to produce recognition of the appropriate pattern. These results have shown that this same conclusion can be drawn by removing intermediate layer neurons, and observing the increased errors.

Part 4: Conclusions

> *This is not the end. It is not even the beginning of the end. But it is, perhaps, the end of the beginning.* ***Winston Churchill, 1942***

Chapter 9: Conclusions

The result of this thesis has been the production of a portable neural network simulator called fishNET that will operate with networks of any size, limited only by available memory and processing speed.

Here, we examine some of the factors important in using the program, namely performance and flexibility. Also, we recap some uses for fishNET, and the results of parameters and casualties on network behaviour. Lastly, some avenues for possible further work are suggested.

9.1. Software – Design and Performance

A large part of the work for this thesis involved writing and testing a software package called fishNET. FishNET was written to be a portable, flexible, network simulator, and was ported to several installations without change.

The design decisions and their end result, the performance of the product, have been examined in detail. The large number of floating point calculations required during learning tends to be the limiting factor involved with using fishNET, not a lack of memory, especially in PC implementations.

9.2. Possible Uses of FishNET

Briefly, the flexibility of neural networks in general was examined by testing fishNET with a wide range of problems, for which it successfully generated generalised networks. The fact that the same program can be quickly "taught" to perform almost any pattern recognition task is an important result. Hence, neural networks are an important tool for engineers.

9.3. Behaviour of Neural Networks

The behaviour of neural networks with respect to parameter variations and casualties was investigated. The problem of dot-matrix encoded English characters was studied in considerable detail.

The results of these investigations showed that there can be large variations in learning times, depending upon the learning parameter ε and the number of neurons in the middle layer. In summary, learning time is shortest for values of ε of between 0.3 and 0.5. Also, for these ε values, the learning time decreases slightly as the number of middle layer neurons increases. However, for larger values of ε (of between 0.9 and 1.0), the learning time *increases* as the number of neurons in the intermediate layer gets larger, and a very wide range of learning times was experienced between runs for these values. This was especially true for $\varepsilon = 1.0$, which appears to be on the boundary of convergence for large numbers of middle layer neurons.

Error rate was also found to be partly dependent upon ε and neurons in the middle layer, although not as strongly as learning time. In fact, for all the experiments with parameter variation, variance in error rate was only about 20%. However, larger values of ε showed lower error rates, and this seems to be a function of the learning time being longer in these cases. Longer learning times logically should return smaller errors, and this was indeed shown to be the case.

Other important discoveries of network behaviour came from introducing casualties of two types and varying severity into the network. First, random damage to connections (weights) had almost no appreciable impact for casualty rates up to between 20% and 30% of the total weight population. Errors increased exponentially beyond 30% casualties. This shows that the knowledge of any pattern

is distributed amongst weights in the network.

Second, random removal of middle layer neurons, or the systematic removal of weights, had more dramatic effects. When 2 out of 15 middle layer neurons were disabled, performance was still very good. However, once another neuron was removed, performance degraded rapidly. Since the number of weights systematically removed in this way was still much smaller than with random removal, but the effect was greater, an important conclusion can be drawn. The knowledge of features of patterns in a neural network is localised or partly localised into certain neurons in the middle layer. Or, expressing this is another way, certain neurons in the middle layer act as feature extraction devices. This was widely known, but to this author's knowledge had only been deduced by looking at the structure of weights. Here it has been done without examining the distribution and strength of connections, instead by viewing performance with casualties in the network.

9.4. Further Work

As a large part of the time involved with this thesis was required to write fishNET, there is still a lot of work that could be done using the developed software tool. Some ideas for potential enhancements, and further work, are listed below.

With fishNET itself, a faster processor such as an 80286/7 or 80386/7 with still a reasonably small memory (say under 2 Mbytes) would make much larger simulations possible. To save space, all variables of type `double` could be changed to `float`, to save almost 50% from memory usage. This wasn't done because there was no point for this first version, where for reasonably small simulations, speed, not space, was the limiting factor.

Another improvement to fishNET would be a graphical interface, that could display outputs during learning (although this would slow things up even more), and connection strengths both during learning and execution. This should be reasonably easy, and would probably involve just changing the output routines used at present to display graphical instead of numeric data. (That was the intention of this first version of fishNET.)

More experiments, especially with regard to error rate, are desirable. A wider range of parameters and several simulations for each set of parameters would remove the statistical "hiccups" from the data, similar to the results for learning rate, which are much better due to averages being used.

Removing neurons in the middle layer offers promise as a mechanism for discovering the structure of feature selection in a back-propagation neural network. More work here could lead to insights into which features are extracted, by examining behaviour with the individual patterns.

Finally, and most difficult, would be a truly parallel processing version of the

back-propagation algorithm, aimed at operation in an n-transputer environment.

—THE END —

Part 5: Appendices

Appendix A: Software

This appendix contains complete printouts of all source and include files used with the package fishNET.

'C' Source Files

fishNET.c

[Listing: fishNET.ct]

error.c

[Listing: error.ct]

learn.c

[Listing: learn.ct]

input.c

[Listing: input.ct]

show.c

[Listing: show.ct]

'C' Include Files

fishNET.h

[Listing: fishNET.ht]

error.h

[Listing: error.ht]

learn.h

[Listing: learn.ht]

input.h

[Listing: input.ht]

show.h

[Listing: show.ht]

net_type.h

[Listing: net_type.ht]

Makefile

[Listing: fishNET.t]

The Help File — help.hlp

[Listing: help.hlp]

Appendix B: Dot-Matrix Encoded Characters

This appendix contains the data used for dot-matrix encoding of English language characters. Input and expected output are shown

Input Data

[Listing: hugein.dat]

Expected Output Data

[Listing: hugeout.dat]

Data Used for Testing

The data below was used for all the experiments in the thesis.

[Listing: hugerun.dat]

References

[1] Robert Hecht-Nielsen, "Neurocomputing: picking the human brain," *IEEE Spectrum* **25** no 3 (1988), 36–41.

[2] David W. Tank and John J. Hopfield, "Collective Computation in Neuronlike Circuits," *Scientific American* December 1987, 62–70.

[3] Mark F. Bear, Leon N. Cooper, and Ford F. Ebner, "A Physiological Basis for a Theory of Synapse Modification," *Science* ***237*** (1987), 42–48.

[4] John J. Hopfield and David W. Tank, "Computing with Neural Circuits: A Model," *Science* ***233*** (1986), 625–633.

[5] David Zipser and Richard A. Andersen, "A back-propagation programmed network that simulates response properties of a subset of posterior parietal neurons," *Nature* ***331*** (1988), 679–684.

[6] Donald Hebb, "The organization of behaviour," John Wiley, New York (1949).

[7] H. Frohn, H. Geiger, and W. Singer, "A Self-Organizing Neural Network Sharing Features of the Mammalian Visual System," *Biological Cybernetics* ***55*** (1987), 333–343.

[8] Ralph M. Seigel, "Discovering structure from motion in monkey, man and machine," from *Neural information processing systems* D.Z.Andersen ed. (1988)

[9] Gail A. Carpenter and Stephen Grossberg, "The ART of Adaptive Pattern Recognition by a Self-Organizing Neural Network," *IEEE Computer* March 1988, 77–88.

[10] Teuvo Kohonen, "The 'Neural' Phonetic Typewriter," *IEEE Computer* March 1988, 11–22.

[11] Kunihiko Fukushima, "A Neural Network for Visual Pattern Recognition," *IEEE Computer* March 1988, 65–75.

[12] Jerome A. Feldman, Mark A. Fanty, and Nigel H. Goddard, "Computing with Structured Neural Networks," *IEEE Computer* March 1988, 91–103.

[13] James A. Anderson, "Networks for fun and profit," *Nature* ***322*** (1986), 406–7.

[14] A paper by Amir F. Atiya, "Learning on a general network," Department of Electrical Engineering, California Institute of Technology CA 91125 — no other publishing data.

[15] Alfred V. Aho, John E. Hopcroft, and Jeffrey D. Ullman, "Data Structures and Algorithms," Addison-Wesley, Sydney, 1983.

[16] Alan F. Murray and Anthony V. W. Smith, "Asynchronous VLSI Neural Networks Using Pulse-Stream Arithmetic," *IEEE Journal of Solid-State Circuits* ***IEEE-JSSC 23*** no 3 (1988), 688–697.

[17] John J. Hopfield and David W. Tank, "'Neural' Computation of Decisions in Optimization Problems," *Biological Cybernetics* ***52*** (1985), 141–152.

[18] Ralph Linsker, "Self-Organization in a Perceptual Network," *IEEE Computer* March 1988, 105–117.

[19] A paper by G. Z. Sun, H. H. Chen, and Y. C. Lee, "Learning Stereopsis with Neural Networks," Laboratory for Plasma and Fusion Energy Studies, Dept. of Physics and Astronomy, and Inst. for Advanced Computer Studies, all University of Maryland, College Park MD 20742 — no other publishing data.

[20] David E. Rumelhart, Geoffrey E. Hinton, and Ronald J. Williams, "Learning representations by back-propagating errors," *Nature* ***323*** (1986), 533–536.

[21] Richard F. Lyon and Carver Mead, "An Analog Electronic Cochlea," *IEEE Transactions on Acoustics, Speech, and Signal Processing* ***IEEE-ASSP 36*** no 7 (1988), 1119–1134.

[22] Jacques J. Vidal, "Implementing Neural Nets with Programmable Logic," *IEEE Transactions on Acoustics, Speech, and Signal Processing* ***IEEE-ASSP 36*** no 7 (1988), 1180–1190.

[23] R. Paul Gorman and Terrence J. Sejnowski, "Learned Classification of Sonar Targets Using a Massively Parallel Network," *IEEE Transactions on Acoustics, Speech, and Signal Processing* ***IEEE-ASSP 36*** no 7 (1988), 1135–1140.

[24] Yi-Tong Zhou, Rama Chellappa, Aseem Vaid, and B. Keith Jenkins, "Image Restoration Using a Neural Network," *IEEE Transactions on Acoustics, Speech, and Signal Processing* ***IEEE-ASSP 36*** no 7 (1988), 1141–1151.

[25] David J. Burr, "Experiments on Neural Net Recognition of Spoken and Written Text," *IEEE Transactions on Acoustics, Speech, and Signal Processing* ***IEEE-ASSP 36*** no 7 (1988), 1162–1168.

[26] Alan S. Gevins and Nelson H. Morgan, "Applications of Neural-Network (NN) Signal Processing in Brain Research," *IEEE Transactions on Acoustics, Speech, and Signal Processing* ***IEEE-ASSP 36*** no 7 (1988), 1152–1161.

[27] Thomas Plum and Jim Brodie, "Efficient C," Plum Hall, Cardiff New Jersey, 1985.

[28] John E. Angus, "On Computing MTBF for a k-out-of-n : G Repairable System," *IEEE Transactions on Reliability* ***37*** no 3 (1988), 312–313.

[29] Donald E. Knuth, "The TeXbook," Addison Wesley, Sydney, 1986.

Acknowledgements

There are many people that I would like to thank for making my work so much easier and enjoyable.

First and foremost, I would like to thank my supervisor, Dr Peter Nickolls. Thank you for always providing sound advice, for letting me run the project as I saw fit, and for steering me in the right direction with regards to which model to work on.

Thank you to my family for supporting me and being tolerant when I got up late. Thank you all for proofreading both this thesis and my essay, for providing the computer, and for feeding me.

Thanks to the programmers at Bain and Company's Fixed Interest division, (Jodi, Phil who came to the seminar, and Monique,) but especially Dr Keith Brinck and Robert Gambi for so kindly letting me use their Pyramid. Without those facilities, I would certainly have gone mad waiting for network convergence, and would not have so many experimental results. Thanks also for helping me set up in your installation, and for the 'C' style advice.

To Kerryn, thanks for being interested above and beyond the call of duty, and to my friends for all the wit, wisdom, and good times over the past five years.

Thank you Andrew Joyner for thinking of the name fishNET for the package, without which it wouldn't have been the same.

And finally, thanks to my brother Alastair, who kept me ideologically sound during all those 2 a.m. tea breaks.